

tasks

Chapter Hierarchy — Implementation Plan

TASK IDS ARE PER-FEATURE AND ARE NOT GLOBALLY UNIQUE — CITE THEM QUALIFIED. This file, `specs/001-workshop-curriculum-platform/tasks.md` and `specs/002-knowledge-areas-deep-linking/tasks.md` each number their tasks independently from T001, so **every id in this file — T001 through T037 — already names two other unrelated pieces of work.** 002's own header records that its T001–T120 collide with 001's in full; this file adds a third meaning to the first 37 of them.

The requirement ids collide the same way. FR-001–FR-035 and SC-001–SC-025 are defined in all three specifications with different meanings. This feature's SC-009 measures a test's blindness to a suffix defect; 002's SC-009 is a media-backed citation landing inside its span; 001's SC-009 is citation support over generated answers. **Nothing here is unique on its own.**

Rule — prefix the feature number: 003:T013, 003:SC-009, 003:FR-012. A bare id in a brief, a commit message, a review or a source comment is ambiguous by construction, and a bare id has already been resolved to the wrong task at least once in this repository.

Re-derive before citing, never from memory:

```
grep -n '^-\ *\*SC-009\*\*' specs/00{1,2,3}-*/spec.md
```

Ids are NOT renumbered to fix this. Renumbering invalidates every existing cross-reference at once. Qualification at the point of citation is the remedy.

For agentic workers: REQUIRED SUB-SKILL: use
superpowers:subagent-driven-development (recommended) or
superpowers:executing-plans to implement this plan task-by-task.
Steps use checkbox (- []) syntax for tracking.

Goal: make a sub-chapter a chapter. One id grammar, five roles, every hierarchy field derived from the id and none of them stored — and every place that quietly assumed a flat list either fixed or made loud.

Architecture: the canonical chapter id is the dotted, zero-padded numeric path (`^[0-9]{2,}(\.[0-9]{2,})*$`), used unchanged as the directory name, the URL segment, the passage registry scope, the `chapter_slug` and the `chapter-` suffix. **Four of the five layers it must survive already survive it** — measured, not assumed. `parent_id`, `depth`, `ancestor_ids`, `child_ids` and `ordinal_path` are derived by string operation and never stored, so they cannot disagree with the id. Ordering is byte-lexicographic on the id and is already implemented correctly; the grammar, not the comparator, is what makes it true.

Tech stack: Go 1.26.2 · Angular 19.2 standalone · Python 3.14.6 + Bash.
No new dependency, no new route, no new column, and exactly one new package (`pkg/chapterid`).

Global constraints

Every task's requirements implicitly include this section. Values are copied verbatim from `spec.md`, `data-model.md` and `research.md` — do not round them and do not “improve” them.

- **Fix what hides first.** The phase order is by how loudly a defect fails, quietest first. The three already repaired in 266f443 all exited 0 while being wrong.
- **The id grammar is `^[0-9]{2,}(\.[0-9]{2,})*$`.** Depth unbounded (H6). 02.2 and 2 are refused — that refusal is the mechanism that makes the sort order true.
- **Ordering is byte-lexicographic on the id string**, verified by execution: 01 02 02.01 02.01.01 02.09 02.10 03 10. . is **0x2E**, digits are **0x30–0x39**.
- **`cmd/workshop-server/main.go:2126` MUST NOT CHANGE.** It is already the correct comparator. The assertion goes on the ordering, not on the code.
- **Every hierarchy field is DERIVED and NONE is stored (H2).** No column, no file, no cache.

- **Every chapter-scope comparison is EQUALITY** (H7). HasSuffix/HasPrefix/Contains on a chapter scope is a defect.
- **A one-scope fixture cannot see the suffix defect.** HasSuffix and == agree on every input a one-chapter registry can construct, so the narrowing fix is a **no-op** there. Fixture shape is the guard.
- **No sub-chapter-specific code path anywhere** (H5). if depth > 1 is a defect.
- **No new route** (FR-026). The manifest already carries **20** NOT_BUILT rows.
- **The chapters array stays FLAT.** core/api.ts:88 reads it flat.
- **Every check is three-valued:** 0 fine · 1 a real problem found · 2 could not determine. A missing dependency, unreachable service or crashed helper is always 2.
- **Every check owes a paired mutation proof.** The widenings owe **two** mutations each, one per direction.
- **Every gate takes its chapter from the live tree**, following verify-absence-honesty.sh:127–134.
- **No server-side CI anywhere in the fleet.**
- **Do NOT run a gate with --root /nonexistent or --prove-failure while exploring.** Some gates write into tracked evidence files and a speculative invocation corrupts the tree.
- **ONE clarification is open and it BLOCKS Phase 1 onward.** The ordinal type (specs/001-.../data-model.md:45) and the path-key contradiction (specs/001-.../contracts/http-api.md:60). Tasks carrying [BLOCKED: ordinal type] are waiting on an operator, not on an implementer.

Task format

[ID] [markers] [Phase] Description

Markers: [P] parallelisable · [TDD] test-first · [REVIEW] review before proceeding · [BLOCKED: x] waiting on a named decision, not on work.

Numbering is permanent. Ids are never reused or renumbered — other documents cite them.

Standing rules

1. **An assertion that greps for a string is not a test.** It must execute the behaviour and check the observable result.

2. **A widening owes a proof in BOTH directions.** A proof asserting only that the new form matches passes against a pattern that has stopped matching the old one — a worse defect, equally silent.
 3. **A narrowing owes a fixture that can see it.** For H7 that means at least two scopes, one a sub-chapter of the other.
 4. **A default that names a real chapter is indistinguishable from a choice.** Where an input is required, refuse; do not choose.
 5. **workshop/ is PRIVATE and specs/ is PUBLIC.** Refer to workshop files by path; quote code — identifiers, regexes, route strings, field names — and never prose, transcript text, curriculum content, material filenames or personal names.
 6. **Never edit a shell script while it is executing.**
-

Phase 0: The specification decision

Purpose: settle the two inherited contradictions that every later task rests on. **This phase writes no code.** Nothing in Phase 1 onward may begin until it is approved, because implementing an answer is how the decision gets made by accident.



T001 [REVIEW] [BLOCKED: ordinal type] Record the decision on Chapter.ordinal in a decision record under specs/003-chapter-hierarchy/, and amend specs/001-workshop-curriculum-platform/data-model.md:45. Three options, each costed in research.md D-CH-5: widen to an ordered ordinal_path; keep int and give sub-chapters no ordinal; keep int and flatten into one number space. **The zero-diff option is the only one that makes the defect permanent** — record which was chosen and why the other two were not (FR-022) — AUDIT 2026-09-08: **GENUINELY OPEN — AND THE DECISION WAS TAKEN IN CODE ANYWAY.** No decision record exists under specs/003-chapter-hierarchy/, and specs/001-workshop-curriculum-platform/data-model.md:45 still reads | ordinal | int |. Meanwhile the server already serves ordinal_path: [2,1] beside a deprecated ordinal: 2. This is the outcome the task warned about: implementing an answer is how the decision got made by accident. **Highest-blocking open item in this feature.**



T002 [REVIEW] [BLOCKED: ordinal type] Amend specs/001-workshop-curriculum-platform/contracts/http-api.md:60, which says the zero-padded ordinal is “*not accepted as a path key — one key, one meaning*” while workshop/platform/gates/route-manifest.tsv:72 declares the live route GET /api/chapters/01 —

which is exactly that string. Amend it to name the **dotted id** as the single path key, or record the alternative (a separate slug, and therefore two keys for one chapter) (FR-002) — AUDIT 2026-09-08: GENUINELY OPEN — specs/001-workshop-curriculum-platform/contracts/http-api.md:60 is UNAMENDED and still says the zero-padded ordinal is “**not** accepted as a path key”, while GET /api/chapters/02.01 serves 200 on exactly that string.



T003 [REVIEW] Confirm the two amendments agree with each other and with the live route set (main.go:547, :817–:819, :841, :843) before anything consumes them. Both amendments are to **publish** artifacts of another feature; the diff is reviewed as a governance change, not as a spec edit (FR-002, FR-022) — AUDIT 2026-09-08: GENUINELY OPEN — blocked on T001 and T002, neither of which was performed.

Checkpoint: the blocking decision is recorded and both 001 artifacts are amended. **Stop for human approval.**

Phase 1: Foundational — the grammar, the derivations, the ordering

Purpose: write the definition every later assertion is checked against. **This phase fixes nothing.** It builds the one thing that makes “is this chapter id valid” and “what is its parent” a question with a single answer.



T004 [TDD] [REVIEW] Create workshop/platform/backend/pkg/chapterid/ — **pkg/**, **not internal/**, because the pipeline’s checks and the gates must assert against the *same* definition, and Go’s internal / forecloses that by language rule. It depends on the standard library only, which is what makes it testable with no tree present (FR-001, FR-006) — EVIDENCE: DONE-UNTICKED — package exists as workshop/platform/backend/pkg/curriculum/chapterid.go (NOT the pkg/chapterid/ path this task names). It is under pkg/, which is the requirement the path was chosen for, and it is stdlib-only. go test ./pkg/curriculum/ PASSES (in_process). DEVIATION RECORDED: the name differs from this task; do not create a second package.



T005 [TDD] Implement and prove the grammar $^{[0-9]\{2,\}(\backslash.[0-9]\{2,\})^*\$}$ in `pkg/chapterid/grammar.go`. Gate **G-CH-1**, table-driven: **accepts** 01 02 02.01 02.01.01 100; **rejects** 2 02.2 2.1 02. .01 "". **Paired mutation**: relax to $[0-9]^+$ — every rejection must fail. 02.2 is the input this whole gate exists for: it sorts *after* 02.10 and no sorting test with well-formed input can see it (FR-001, FR-003, FR-005, FR-034, SC-002) — EVIDENCE: DONE-UNTICKED — ChapterIDGrammar = $^{[0-9]\{2,\}(\backslash.[0-9]\{2,\})^*\$}$ at `pkg/curriculum/chapterid.go:58`; table-driven TestGCH1_Grammar (`chapterid_test.go:26`) accepts 01 02 02.01 02.01.01 100 100.01 02.09 02.10 10 and rejects the unpadded forms. `go test ./pkg/curriculum/` PASSES (in_process). Paired mutation is DOCUMENTED at `chapterid_test.go:18` but NOT executed.



T006 [TDD] [BLOCKED: ordinal type] Implement ParentID, Depth, AncestorIDs and OrdinalPath in `pkg/chapterid/derive.go` — **pure string operations on the id alone** (data-model.md §2.2). Table includes the root case: no . $\Rightarrow$ parent_id null, depth 1, ancestor_ids []. **These fall out; they are not special-cased** (FR-008). OrdinalPath is blocked on T001 (FR-006, FR-008, FR-022, SC-012) — EVIDENCE: DONE-UNTICKED — Depth (:101), ParentID (:109), AncestorIDs (:119), OrdinalPath (:136) in `pkg/curriculum/chapterid.go`; TestDerivations_FallOutOfTheID (`chapterid_test.go:155`) covers the root case. `go test ./pkg/curriculum/` PASSES (in_process). NOTE: implemented despite [BLOCKED: ordinal type] — the T001 decision was made IN CODE, not in a decision record.



T007 [TDD] Implement ChildIDs and Orphaned over the **set** of present ids. These are the only two derivations needing more than one id, and they are still derivations — computed on demand, never stored. Name that seam in the code: it is where “just cache the children” will be proposed, and it is the forbidden second representation wearing a performance argument (FR-006, FR-007, FR-008, FR-031) — EVIDENCE: DONE-UNTICKED — ChildIDs (:191), MissingAncestorIDs (:204), Orphaned (:225) in `pkg/curriculum/chapterid.go`; TestChildIDsAndOrphaned (`chapterid_test.go:201`). `go test ./pkg/curriculum/` PASSES (in_process).



T008 [TDD] Prove **G-CH-2** in `workshop/platform/gates/verify-chapter-hierarchy.sh`: over a fixture tree holding 01 02 02.01 02.01.01 02.09 02.10 03 10, API order, shell-glob order and filesystem byte order are **identical, element by element**. **main.go:2126 MUST NOT BE CHANGED by this task** — it is already correct; the gate is what stops a future rewrite. **Paired mutation**: sort by or

dinal_path numerically — must go red on 02.09 / 02.10 (FR-009, FR-010, FR-011, FR-034, SC-003, SC-004) — AUDIT 2026-09-08: PARTIAL — TestGCH2_ComparatorIsUnchangedAndTheOrderIsRight (cmd/workshop-server/chapters_hierarchy_test.go:298) and TestOrdering_ByteLexicographicIsTheWholeRule (chapterid_test.go:62) PASS, and main.go was not changed. MISSING: workshop/platform/gates/verify-chapter-hierarchy.sh does not exist, so the three-way comparison of API order vs shell-glob order vs filesystem byte order is NOT made, and the paired mutation is documented (chapterid_test.go:58) but not executed.



T009 [TDD] Prove **G-CH-3**: no persisted parent_id, depth or child list exists in the registry schema, the derived database, or any serialised artifact. **Paired mutation**: persist one — the gate must go red. This is H2 made checkable rather than asserted (FR-007, FR-034, SC-011) — AUDIT 2026-09-08: PARTIAL — the derivation is pure and unstored by construction (pkg/curriculum/chapterid.go header, “DERIVED, NEVER STORED”), and TestChapterHierarchy_DerivedFromTheIDAlone (internal/api/chapters_hierarchy_test.go:88) PASSES. MISSING: no gate asserts the ABSENCE of a persisted parent_id/depth/child list across the registry schema, derived database and serialised artifacts, and no mutation persists one.



T010 [TDD] Implement **H1** in cmd/workshop-server/main.go:2094 (listChapters): a directory that does not match the grammar is returned in an unclassified list **with a reason**, never silently skipped and never silently listed as a chapter. Contract §4.1 C4.1.7. **Paired mutation**: continue on a non-match — the gate must go red. Preserve the existing discipline at :2111–:2123: material entries are **counted, never named** on the wire (FR-004, FR-034, SC-001) — EVIDENCE: DONE-UNTICKED — ScanChapterDirs (:293) + ClassifyUnclassified (:258) return a reason per rejected directory; TestGCH1_UnclassifiedIsReportedNotSkipped (cmd/workshop-server/chapters_hierarchy_test.go:196) PASSES (in_process), with the paired mutation documented at :194. LIVE: GET /api/chapters 200 carries "unclassified": [] (served).



T011 [P] [TDD] Assert the four embeddings measured in research.md D-CH-2, each of which works today and each of which is one edit from not working: SafeSlug (pkg/curriculum/curriculum.go:205) accepts . and still rejects /; ChapterDir (:187) resolves chapter-02.01; the {chapter} wildcard matches a dotted id as one segment; HasSourceFilenameShape (pkg/search/catalog.go:368) does **not** classify 02.01 as a filename. **The fourth is the important one**

— a false positive there raises a content-boundary alarm on every sub-chapter id, and the fix for it would be one entry in the extension list at `catalog.go:320–323` (FR-002) — AUDIT 2026-09-08: PARTIAL — 2 of the 4 embeddings are asserted: `SafeSlug` accepts `02.01` and rejects a path separator, and the grammar refuses `02.2` (`TestSafeSlug_IsNotTheChapterGrammar`, `chapterid_test.go:299`). MISSING all three of: `ChapterDir` resolving `chapter-02.01`; the `{chapter}` wildcard matching a dotted id as ONE segment; and **the important one** — no assertion that `HasSourceFilenameShape` does not classify `02.01` as a filename. (Inspected: `SourceFilenameShapeExtensions` at `pkg/search/catalog.go:320` contains no numeric entry, so it holds in fact today — it is simply unguarded.)



T012 [REVIEW] Review the grammar and the derivations before anything consumes them. A grammar changed after adoption invalidates every assertion written against it (FR-001, FR-006) — AUDIT 2026-09-08: UNVERIFIABLE — no review record found under `specs/003-chapter-hierarchy/`. Resolved by a recorded review, or by the reviewer stating it happened.

Checkpoint: there is one definition of a chapter id and one derivation of its ancestry. **Stop for human approval.**

Phase 2: The redaction defect — the fix landed, the test did not

Purpose: the quietest defect in the feature. `cmd/workshop-redact/main.go:616` is now `*rec.Scope != chapter`, fixed in 266f443 from `strings.HasSuffix(*rec.Scope, chapter)`. **Nothing guards it.** On a one-scope registry the two operators agree on every constructible input, so the narrowing is a no-op there and every existing test passes with either. Its output is a **disclosure-control record** — a document whose purpose is to state what was covered — which is why it comes before the content defects in Phase 3.



T013 [TDD] Build a **two-scope** fixture registry under the redaction command's test data: scopes `01` and `02.01`, each carrying at least one artifact. The fixture is the guard — `research.md` D-CH-6 tabulates why: for `chapter="01"`, `HasSuffix` and `==` disagree only on the `02.01` row (FR-013, SC-008) — EVIDENCE: DONE-UNTICKED — two-scope fixture (`01` and `02.01`) in `workshop/platform/backend/`

cmd/workshop-redact/scope_test.go:29. go test ./cmd/workshop-redact/ PASSES (in_process).



T014 [TDD] Prove **G-CH-6**: the redaction review for 01 covers exactly the 01 artifacts and **excludes** 02.01's. **Paired mutation**: restore strings.HasSuffix — the test must go red. Without the fix, chapter 01's review absorbs sub-chapter artifacts and the record **overstates its own coverage** (FR-012, FR-013, FR-034, SC-008) — EVIDENCE: DONE-UNTICKED — TestScopeIsChapter_SubChapterIsNotItsParent (scope_test.go:20); the equality narrowing is at cmd/workshop-redact/main.go:616 with the defect recorded at :579 and :615. go test ./cmd/workshop-redact/ PASSES (in_process).



T015 [TDD] Prove **G-CH-6b**, and read this one carefully because it asserts a **negative about a test**: run the same HasSuffix mutation against a **one-scope** fixture and assert the test **stays green**. It has no mutation of its own — it is the mutation. Without it, a maintainer who simplifies the fixture sees everything still pass and concludes the two-scope shape was incidental. It was not (003:SC-009, FR-013, FR-034) — EVIDENCE: DONE-UNTICKED — TestScopeIsChapter_ProvesTheTableCatchesTheDefect (scope_test.go:104) EXECUTES the pre-fix HasSuffix predicate in-binary (:110) and fails if the mutation is inoperative (:116). This is the only EXECUTING paired mutation found in the feature. go test ./cmd/workshop-redact/ PASSES (in_process).



T016 [TDD] Implement **G-CH-5** in verify-chapter-hierarchy.sh: no comparison anywhere in the tree applies HasSuffix, HasPrefix or Contains to a chapter-scope value (H7). **Paired mutation**: introduce one — the gate must go red. This is written tree-wide rather than file-specific because workshop-redact is not the only place a scope is compared, and the next one will look just as harmless (FR-012, FR-034, SC-010) — AUDIT 2026-09-08: GENUINELY OPEN — workshop/platform/gates/verify-chapter-hierarchy.sh does not exist, so G-CH-5 is not implemented anywhere. The one narrowing that WAS made (workshop-redact) is guarded by its own test; nothing checks the tree for the next one.

Checkpoint: the disclosure-control record's coverage claim is guarded by a fixture that can see the defect, and by a proof that the cheaper fixture cannot. **Stop for human approval.**

Phase 3: Silent data loss

Purpose: every defect here produces **plausible output**. None fails. They are ordered within the phase by what a wrong run produces (research.md D-CH-8), worst first.



T017 [TDD] Prove **G-CH-13. THE CODE FIX LANDED CONCURRENTLY — THE PROOF DID NOT, AND THAT IS WHAT THIS TASK IS**. Re-measured at the end of the drafting session (spec §“The tree moved DURING drafting”): another agent replaced the frozen transcript default with a derived one at `workshop/scripts/ingest.sh:120` (`full_ch${_CH_FLAT}`, dots stripped) and added a **refusal** at `:109` routed through the three-valued undetermined path. **No test appeared with it**. Assert: naming a chapter with no transcript export refuses, names the missing input, and writes **zero** records. **Paired mutation:** restore the frozen `full_ch01` default — the gate must go red. Until this exists, the fix is protected by nothing and reverts the first time the refusal is found inconvenient (FR-016, FR-034, SC-007) — AUDIT 2026-09-08: GENUINELY OPEN AS A PROOF — the code fix IS present (`workshop/scripts/ingest.sh` derives `_CH_FLAT="${CHAPTER//./}"` and `TRANSCRIPT_JSON` from it, with the refusal above it). **No test appeared with it**, and none has since: no test exercises the refusal, names the missing input, or asserts zero records written. The fix is still protected by nothing.



T018 **DONE IN CODE, NOT BY THIS FEATURE — verify and close, do not reimplement**. `workshop/scripts/ingest.sh:46` was `CHAPTER="01"`; it now reads `CHAPTER=""` at `:60` with a required-argument check at `:107–:111`. Confirm the message names both forms an operator might pass (`02` and `02.01`) and that the exit is the three-valued undetermined, not a plain failure. **Do not re-open the default** (standing rule 4) (FR-017) — EVIDENCE (source): DONE-UNTICKED (verify-and-close) — `workshop/scripts/ingest.sh:60` reads `CHAPTER=""`; the required-argument check names BOTH `02` and `02.01` and routes through the three-valued undetermined, not a plain failure. Default NOT re-opened.



T019 [TDD] Prove **G-CH-12** for the two patterns already widened in 266f443: `_TRANSCRIPT_PATH_RE` at `workshop/pipeline/extract/meeting_notes.py:109` and `_ELIGIBLE_SOURCE` at `workshop/pipeline/extract/author.py:159`. Assert each matches `chapter-01/transcript.md` **and** `chapter-02.01/transcript.md`. **TWO paired mutations, both required:** (a) revert to `chapter-\d+` — the dotted

assertion must fail; (b) narrow to chapter-\\d+\\.\\d+ — the flat assertion must fail. (b) is not redundant: a proof asserting only (a) passes against a pattern that has stopped matching the flat form. **Do not widen exercise-\\d+\\.md** — exercise numbering is not a chapter path (FR-014, FR-034, SC-006) — AUDIT 2026-09-08: GENUINELY OPEN AS A PROOF — both widenings ARE in place: `_TRANSCRIPT_PATH_RE` at `pipeline/extract/meeting_notes.py:109` and `_ELIGIBLE_SOURCE` at `pipeline/extract/author.py:159`, and `exercise-\\d+\\.md` was correctly left alone. **Neither paired mutation exists.** `test_meeting_notes.py` and `test_author.py` exercise only chapter-01 and chapter-02 — no dotted id appears in any pipeline test, so the widening is asserted in NEITHER direction.



T020 Implement FR-015 at both call sites: a pattern non-match on a directory that **exists is reported**, not skipped with `continue`. The two regex fixes closed the instances; the `continue` is the mechanism, and it is still there (FR-015, SC-005) — AUDIT 2026-09-08: GENUINELY OPEN — the `continue` mechanism is still present at `pipeline/extract/meeting_notes.py:126` and `:129` (and at `:198`, `:201`, `:359`, `:404`, `:407`). The regex fixes closed the instances; FR-015 reporting a non-match on a directory that exists is not implemented at either call site.



T021 [P] Derive the suggest source manifest instead of hardcoding it. Six rows in `workshop/platform/backend/pkg/search/suggest-sources.json` — lines 10, 34, 57, 77, 101, 125 — name `chapter-01/...`, and the file is compiled in with `//go:embed` at `pkg/search/catalog.go:70–71`, so **no new chapter reaches /api/suggest without a rebuild**. Preserve the existing override path (`LoadManifestFile`) and the reasoning behind it: a suggest surface that cannot start because a directory moved is a worse failure than one that starts with the kinds it knows about (FR-021) — AUDIT 2026-09-08: GENUINELY OPEN — all six rows in `workshop/platform/backend/pkg/search/suggest-sources.json` still name `chapter-01/...` at lines 10, 34, 57, 77, 101 and 125, unchanged.



T022 [P] Derive every chapter reference in `workshop/curriculum/chapter-01/knowledge/build.py`. Two frozen deep links — `"/chapters/01/transcript#p-%s"` at `:363` and `:435` — and four frozen `"chapter_slug": "01"` literals at `:478`, `:564`, `:582`, `:592`. **The correct form is already in the same file** at `:415`: `"chapter_slug": r["scope"]`. Use it (FR-020) — AUDIT 2026-09-08: GENUINELY OPEN — both frozen deep links (`/chapters/01/transcript#p-%s`) remain at `workshop/curriculum/chapter-01/knowledge/build.py:363` and `:435`,

and all four frozen "chapter_slug": "01" literals remain at :478, :564, :582 and :592. The correct form is still sitting unused at :415.



T023 [P] Remove the frozen defaults at workshop/pipeline/build_transcript.py:143–144 (--chapter default="chapter-01", --title default="Chapter 1"). Both arguments become required, or both are derived from the transcript being built (FR-018) — AUDIT 2026-09-08: GENUINELY OPEN — --chapter default="chapter-01" and --title default="Chapter 1" are both still present at workshop/pipeline/build_transcript.py:143-144.



T024 [P] Resolve workshop/pipeline/calibrate.sh:36 from a named chapter instead of a frozen absolute path to chapter 01's recording file. **Do not reproduce the path or the filename in any spec, commit message or test fixture** — it is private-tree content (standing rule 5). Resolve the recording through the chapter directory the script was asked about (FR-019) — AUDIT 2026-09-08: GENUINELY OPEN — workshop/pipeline/calibrate.sh:36 still assigns REC from a frozen absolute path into chapter 01. (Path deliberately not reproduced here: private-tree content, standing rule 5.)

Checkpoint: no stage can process the wrong chapter's content, and no stage can drop a chapter without saying so. **Stop for human approval** — the diff changes what the pipeline writes.

Phase 4: Visible defects

Purpose: wrong answers a reader can see. They get fixed anyway; they do not need to be first.



T025 [TDD] [BLOCKED: ordinal type] Replace ordinalOf (workshop/platform/backend/internal/api/chapters.go:736) with a call into pkg/chapterid. Today it consumes leading digits and returns 2 for both 02 and 02.01. Gate **G-CH-4**: ordinal representations are **pairwise distinct** over every present id plus a synthetic set of at least eight. **Paired mutation:** restore leading-digit parsing — must go red on the 02 / 02.01 pair. **The defect is the int type in specs/001-.../data-model.md:45, not this function** — which is why T001 blocks this (FR-022, FR-034, SC-013) — AUDIT 2026-09-08: PARTIAL — AND THE REQUIREMENT WAS DELIBERATELY NOT MET. ordinalOf (internal/api/chapters.go:783) was NOT replaced; it is kept UNCHANGED and marked DEPRECATED, with OrdinalPath added

beside it for backward compatibility (rationale in the comment at :775). It still returns 2 for BOTH 02 and 02.01. G-CH-4 (ordinal representations pairwise distinct over every present id) is NOT asserted for ordinal — only titles are, by `TestTitle_DoesNotCollide`. This is a design decision that contradicts the task text; it needs recording, not reimplementing.



T026 [BLOCKED: ordinal type] Fix `chapterTitle` (`chapters.go:755`), which renders 02 and 02.01 both as "Chapter 2". Two distinct chapters with one title is a wrong answer a reader will misread as a cosmetic quirk (FR-023) — EVIDENCE: DONE-UNTICKED — `chapterTitle` (`internal/api/chapters.go:809`) is now a one-line forward to `curriculum.ChapterID.Title`; `TestTitle_DoesNotCollide` (`chapterid_test.go:119`) asserts `Title("02.01") != Title("02")`. LIVE: GET `/api/chapters/02.01` 200 returns "title": "Chapter 2.1" (served).



T027 Rewrite `derivation.ordinal` (`chapters.go:141`) in the same change as T025. It currently reads “*parsed from the leading digits of the chapter slug*”, which is an **accurate description of the defect** — the endpoint is not lying, it is stating exactly what it did. **Rewrite it, do not delete it:** that honesty mechanism is why the defect is legible, and a string left describing a replaced derivation converts a working instrument into a stale one (contract C4.2.1) (FR-024, FR-030) — EVIDENCE: DONE-UNTICKED — `derivation.ordinal` rewritten at `internal/api/chapters.go:172` to describe the collision rather than the replaced derivation, and `derivation.ordinal_path` / `derivation.hierarchy` added beside it. LIVE: GET `/api/chapters/02.01` 200 carries the rewritten strings (served).



T028 [P] [TDD] Reconcile the front end’s two chapter shapes in `workshop/platform/frontend/src/app/core/models.ts`. The minimal shape at :301 is ordinal: `/^\d+$/.test(key) ? Number(key) : null` and yields **null** for 02.01; the full shape at :357 is ordinal: `num(raw['ordinal'])` and yields **2**. **They disagree today.** Assert field-by-field equality for every present chapter. **Paired mutation:** restore the `/^\d+$/.test` — must go red on 02.01 (003:SC-014, FR-025, FR-034) — AUDIT 2026-09-08: GENUINELY OPEN — the two shapes still disagree. `workshop/platform/frontend/src/app/core/models.ts:301` is ordinal: `/^\d+$/.test(key) ? Number(key) : null` (yields null for 02.01) and :357 is ordinal: `num(raw['ordinal'])` (yields 2). `models.spec.ts` exercises no dotted id and asserts no field-by-field equality.



T029 [TDD] Prove **G-CH-11**: `core/api.ts:88` reads `body['chapters']` as a flat array and must keep working unchanged. Assert the JSON type and assert no element carries a nested chapter array. **Paired mutation**: nest children under their parent — the gate must go red (FR-027, FR-034, SC-016) — AUDIT 2026-09-08: GENUINELY OPEN — `core/api.ts:88` does read `body['chapters']` flat and the live response IS flat, but nothing asserts it: no spec in `src/app/core/` checks the JSON type or that no element carries a nested chapter array, and no paired mutation nests children.

Phase 5: The API surface

Purpose: serve what Phase 1 derives, through the two endpoints that already exist. **No route is added** (FR-026, contract §5).



T030 Add the hierarchy object to every row of GET `/api/chapters` — `parent_id`, `depth`, `ancestor_ids`, `child_ids`, `ordinal_path`, `orphaned` — computed from `pkg/chapterid`. **The array stays flat** (T029). Contract §4.1 (FR-006, FR-027) — EVIDENCE (served): DONE-UNTICKED — `chapterHierarchy` at `internal/api/chapters.go:812`. LIVE: GET `/api/chapters` 200 returns per-row hierarchy with `parent_id`, `depth`, `ancestor_ids`, `child_ids`, `ordinal_path`, `orphaned`, and the array is FLAT. Measured 2026-09-08 against 127.0.0.1:8087.



T031 [TDD] Add `under`, `depth` and `include_self`, and echo the applied filters on **every** status including `unavailable`. Gate **G-CH-7**; **paired mutation**: omit the echo on the `unavailable` path. Without it, four situations are indistinguishable to a client — empty branch, misspelled parameter, parameter ignored by an older build, backend unreadable — and the first is a fact while the other three are faults. Reject an unparseable `depth` or `include_self` as a **malformed request** rather than defaulting it, following `boolParam` (`chapters.go:819–821`), which already rejects for exactly this reason (FR-028, FR-034, SC-017) — AUDIT 2026-09-08: GENUINELY OPEN — MEASURED, not inferred. GET `/api/chapters?under=02, ?under=02&depth=2, ?under=99` and `?under=02&include_self=true` all return 200 with the SAME unfiltered three ids, and NO `filters` key is echoed on any of them. The parameters are ignored entirely. **This is the largest open capability in the feature and it blocks T032 and half of T034.**



T032 [TDD] Prove **G-CH-8**: an under naming a chapter that does not exist returns **200** with an empty array and `under_resolved: false` — **never 404**. The route exists and the request was well-formed; a 404 answers “no such endpoint” to a question whose answer is “no such branch”. **Paired mutation**: return 404. Note also that `under_resolve` is null, not false, when `under` was not given or could not be looked for (FR-029, FR-034, SC-018) — AUDIT 2026-09-08: GENUINELY OPEN — `?under=99` does return 200, but only because `under` is ignored (see T031). There is no `under_resolved` field on the response at all, so the distinction between “no such branch” and “parameter not implemented” cannot be made by a client.



T033 [BLOCKED: ordinal type] Add hierarchy with `parent_href` and `child_hrefs` to GET `/api/chapters/{chapter}`, replace ordinal with `ordinal_path` — **replace, never place beside**, or a client reads the one that collides — and carry the rewritten `derivation.hierarchy`. Contract §4.2 (FR-022, FR-024, FR-030) — AUDIT 2026-09-08: PARTIAL — hierarchy with `parent_href` and `child_hrefs` IS served (LIVE: GET `/api/chapters/02.01` 200 returns `parent_href: "/api/chapters/02"`, `child_hrefs: []`, plus `missing_ancestor_ids`), and `derivation.hierarchy` is carried. **BUT the task says “replace ordinal with ordinal_path — replace, never place beside”, and the server places them BESIDE**: the same response carries `ordinal: 2` AND `ordinal_path: [2,1]`. The code states this is deliberate, for one release (chapters.go:147-155). Reconcile with T001/T025 rather than reimplementing.



T034 [TDD] Prove **G-CH-9 (H3)**: a sub-chapter whose parent directory is absent is **served**, with `orphaned: true`, `parent_href: null` and `missing_ancestor_ids` naming what was not found — and `parent_id` **unchanged**, because it is derived. `?under=<missing parent>` still returns it, because the filter runs on derived ancestry. **Paired mutation**: filter orphans out of the list — the gate must go red. Hiding it loses content that exists; promoting it to a root silently rewrites the hierarchy (FR-031, FR-034, SC-019) — AUDIT 2026-09-08: PARTIAL — the serving half is proven: `TestGCH9_OrphanIsServed` (internal/api/chapters_hierarchy_test.go:228) and `TestGCH9_OrphanIsListed` (cmd/workshop-server/chapters_hierarchy_test.go:244) PASS, with the paired mutation documented at :225/:243, and `missing_ancestor_ids` is served. MISSING: the `?under=<missing parent>` half cannot be proven because `under` is not implemented (T031).

Phase 6: Gates, manifest, closure



T035 Update workshop/platform/gates/route-manifest.tsv. Three separate changes: (a) :64 reads #subst {chapter} 01 — **one substitution value cannot exercise a hierarchy**, so substitute at least a parent and a child; (b) the rows written with a literal /api/chapters/01/ — :72–:76, :85, and the four NOT_BUILT rows at :157–:160 — are re-expressed against {chapter} so the substitution reaches them; (c) both changed endpoints carry a **compound** contract citation, 3.2+003.4.1 and 3.3+003.4.2. A row naming only the 001 section still looks complete to a reviewer while the behaviour underneath it has changed — the defect is a **stale** row, not an absent one, and no existence check can see it (FR-033) — AUDIT 2026-09-08: GENUINELY OPEN — none of the three changes is made. workshop/platform/gates/route-manifest.tsv still substitutes one chapter value, rows 71-76 and 85 still carry a literal /api/chapters/01/, the four NOT_BUILT rows are still literal, and no row carries a compound 3.2+003.4.1 / 3.3+003.4.2 citation.



T036 [TDD] Implement **G-CH-10** and **G-CH-14** in verify-chapter-hierarchy.sh, with prove-chapter-hierarchy.sh as its paired prover. **G-CH-10: no code path anywhere branches on a chapter's depth (H5)** — report each hit with its location rather than counting; **paired mutation**: add if depth > 1 to any handler. **G-CH-14**: every gate obtains its chapter from the live tree or from an argument, following verify-absence-honesty.sh:127–134, whose undet on “no chapter could be read” is the third state and must be copied with it; **paired mutation**: hardcode 01 in one gate. Register both in workshop/platform/gates/check-registry-003.tsv (FR-032, FR-034, FR-035, SC-020, SC-024, SC-025) — AUDIT 2026-09-08: GENUINELY OPEN — neither verify-chapter-hierarchy.sh nor prove-chapter-hierarchy.sh exists in workshop/platform/gates/, no G-CH- identifier appears in any gate script or registry, and check-registry-003.tsv does not exist (only check-registry-001.tsv and check-registry-002.tsv).



T037 [REVIEW] Closure: publish max_depth_present as a **measurement** and assert no constant bounds depth (H6, 003:SC-021); record the feature's limits in workshop/docs/limits.md, including that 02 and 02.01 remain **archived and uningested** and that nothing here ingests them; update CONTINUATION.md in the same change; and run the gate-coverage check below to confirm every contracted gate is carried by a task (FR-003, SC-021) — AUDIT 2026-09-08: PARTIAL —

max_depth_present IS published as a measurement (LIVE: GET /api/chapters 200 returns "max_depth_present": 2, computed by MaxDepthPresent, pkg/curriculum/chapterid.go:325). MISSING: workshop/docs/limits.md exists but records nothing about hierarchy, the archived/uningested status of 02 and 02.01, or a no-constant-bounds-depth assertion; and no gate-coverage check confirms every contracted gate is carried by a task.

Dependencies

```

T001 T002 T003          (Phase 0 – BLOCKING, operator decision)
  ↳ T004 → T005 → T006 → T007          (grammar before
derivations)
                                ↳ T008 T009 T010 T011          (gates over the
grammar)
                                ↳ T012 [REVIEW]
                                ↳ T013 → T014 → T015
(fixture before its proofs)
                                |
                                ↳ T016
                                ↳ T017 T018 T019 T020 T021 T022
T023 T024
                                ↳ T025 → T026 T027          (T025
also needs T001)
                                |
                                ↳ T028 T029
                                ↳ T030 → T031 T032
                                |
                                ↳ T033 → T034          (T033
also needs T001)
                                ↳ T035 T036 → T037

```

Hard blocks:

- **T001 blocks T006 (OrdinalPath), T025, T026 and T033.** These carry [BLOCKED: ordinal type] and are waiting on an **operator**, not on an implementer. Writing them settles the decision by shipping, which is the one outcome Phase 0 exists to prevent.
- **T004 blocks everything after it.** One definition of a chapter id, or three.
- **T013 blocks T014 and T015.** The fixture is the guard; writing the assertion first invites a fixture shaped to pass it.
- **T029 blocks T030.** Prove the flat-array contract before adding fields to the rows inside it.

Parallel opportunities

Marked [P]. These share no files.

- **T021, T022, T023, T024** — a Go-embedded JSON manifest, a Python builder, a Python CLI and a shell script. Four languages, four directories.
- **T028** with any Phase 3 task — TypeScript against Python and Bash.
- **T011** with T008/T009/T010 — assertions over four independent packages.

Independent test criteria

Phase	Independently testable when
0	both 001 artifacts are amended and the amendments agree with the live route set
1	pkg/chapterid accepts and rejects its table with no tree present, and the three orders agree over a fixture
2	the two-scope fixture goes red under the mutation and the one-scope fixture stays green
3	a stage run against a tree holding a sub-chapter names it — processed or skipped-with-reason — and ingestion refuses rather than substituting
4	no two chapters share an ordinal representation or a title, and the two front-end shapes agree
5	a client builds the tree from one request, an unknown under is 200, and an orphan is served flagged
6	every contracted gate is carried by a task and every gate reads its chapter from the live tree

Gate coverage — every contracted gate has a task that builds it

Gate	Task
G-CH-1 grammar	T005
G-CH-2 three orders	T008
G-CH-3 derived-only	T009
G-CH-4 no ordinal collision	T025
G-CH-5 scope equality	T016
G-CH-6 two-scope redaction	T014
G-CH-6b fixture blindness	T015
G-CH-7 filters echo	T031
G-CH-8 unknown branch is 200	T032
G-CH-9 orphan served	T034
G-CH-10 no special case	T036
G-CH-11 flat array	T029
G-CH-12 both directions	T019
G-CH-13 ingest refusal	T017
G-CH-14 no hardcoded chapter in gates	T036
H1 unclassified reporting	T010

```
# THE CLOSURE CHECK. A contract gate must be carried by a TASK
    BLOCK — the
# "- [ ] T###" line plus its indented continuation lines. Prints
    one row per
# unattached gate; silence = closed.
awk '/^\\| \\*\\*G-CH-/ {gsub(/\\/|/, " "); print $2}' \
    specs/003-chapter-hierarchy/contracts/http-api.md \
    | tr -d '*' | sort -u \
    | while read -r g; do
        grep -q "$g" specs/003-chapter-hierarchy/tasks.md || echo
        "unattached: $g"
    done
```

Implementation strategy

The whole feature is small, and its risk is not in the code. One new package, one derivation replacement, fourteen frozen literals and a set of gates. The weight is in Phase 0 — two amendments to another

feature's published artifacts — and in Phase 2, where a fix that has already landed is protected by nothing, on a path whose output is a disclosure-control record.

Do not start at Phase 4. The visible defects are the tempting first move: `ordinalOf` returning 2 is a one-line fix that makes a wrong label right, and it feels like progress. It is blocked on Phase 0, and fixing it first would settle the `ordinal` type question by writing an implementation of one answer — which is exactly how `Chapter.ordinal | int` came to be load-bearing in the first place.